

Methods 3: Multilevel Statistical Modeling and Machine Learning

Week 49: *What **was** it all about and Evaluation*

December 2, 2025

Learning goals and outline

Wrap-up of course

1) Understanding how it all fits together

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

EXAM

Submission date: December 15

Methods-3-2025

Methods-3

☆ **Assignments** 5 👤 Students 0 👤 TAs and Admins 1 ⚙️ Settings

Assignments

[assignment_0](#)

● Active 👤 Group assignment for Student teams

[assignment_1](#)

● Active 👤 Group assignment for Student teams

[assignment_2](#)

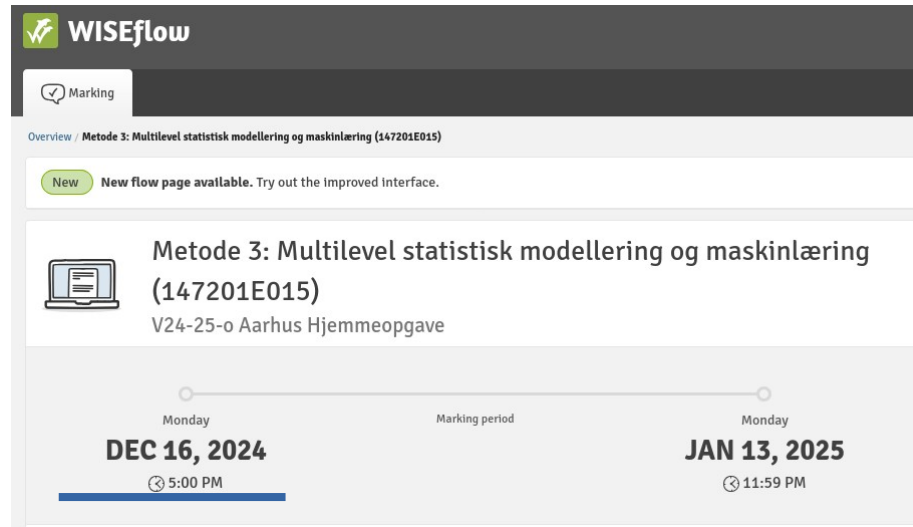
● Active 👤 Group assignment for Student teams

[assignment_3](#)

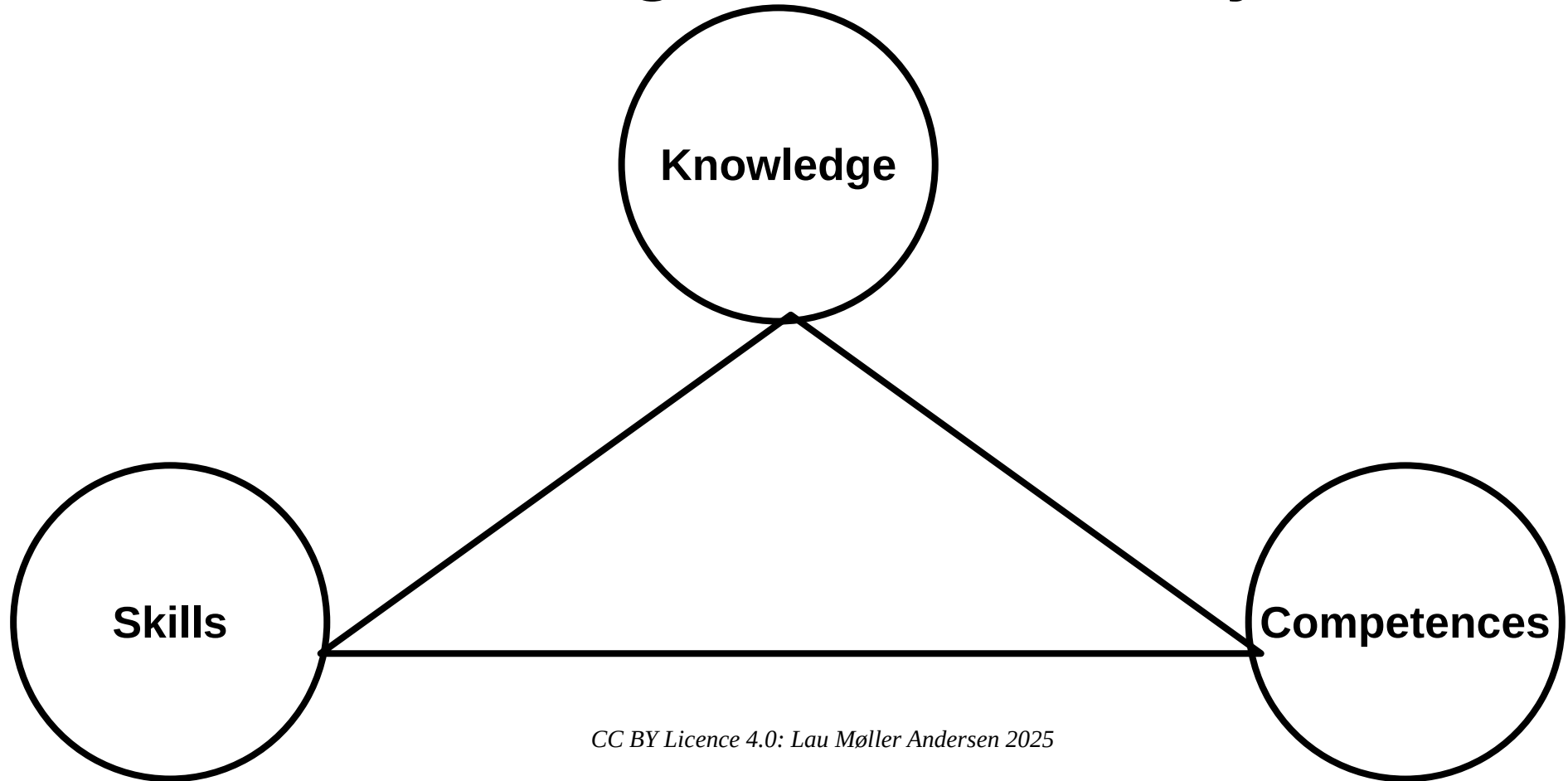
● Active 👤 Group assignment for Student teams

IMPORTANT:

Make sure to submit a knitted *pdf* that shows the plots and the relevant content of the variables.
(Don't go outside the margin; No *Rmd* or *.py* files!)



Academic regulations – objectives



Academic regulations – objectives

Knowledge:

- demonstrate understanding of statistical techniques relying on the Generalised Linear Model
- demonstrate understanding of hierarchical modeling methods
- demonstrate understanding of basic machine learning concepts.

Academic regulations – KNOWLEDGE

You should thus be able to answer questions like:

What is logistic regression?

*What are some naturally
occurring hierarchies within
research data?*

What is a multilevel model?

What is cross-validation?

Academic regulations – objectives

Skills:

- build and evaluate models of hierarchically structured data
- integrate machine learning procedures in data analysis
- communicate analysis processes, results and interpretation.

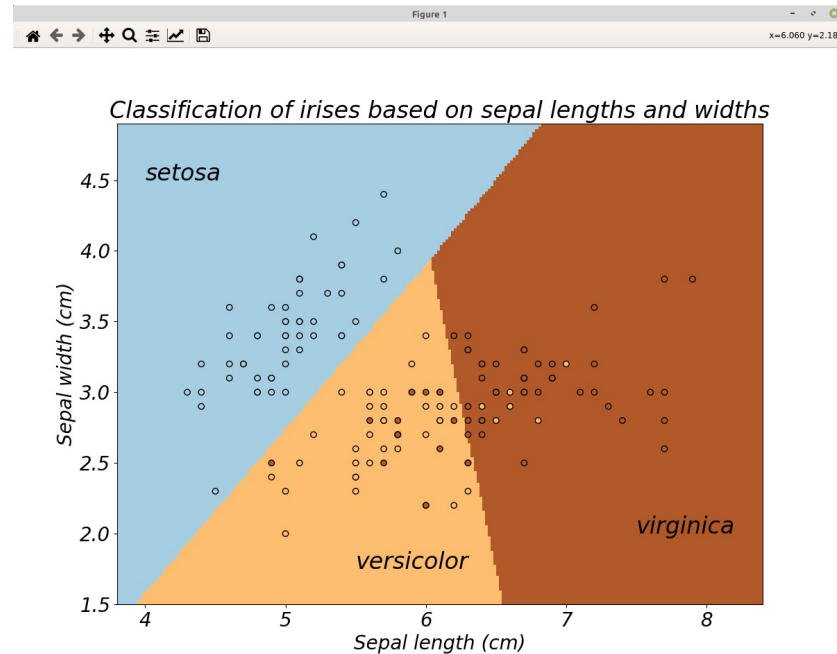
Academic regulations – SKILLS

*build and evaluate models of
hierarchically structured data*

```
Linear mixed model fit by REML ['lmerMod']  
Formula: Height ~ Gender + (1 | Family)  
Data: height
```

Academic regulations – SKILLS

*integrate machine learning procedures
in data analysis*



Academic regulations – SKILLS

*communicate analysis processes,
results and interpretation*

(Andersen et
al., 2019)

Mixed model analyses ([McCulloch & Neuhaus, 2005](#)) were applied to investigate how top-down expectations (No. of Possible Targets) affected subjective experience and objective performance. We performed model comparisons between models that did or did not include the relevant fixed effects and interactions to find the best compromise between an explanatory and a parsimonious model. This was done using the log-likelihood ratio between two models because this ratio approximates a chi-square distribution. A chi-square test can thus be used to assess whether two models differ significantly, where the test statistic is the log-likelihood-ratio and the degrees of freedom is the difference in free parameters of the two models.

Academic regulations – objectives

Competences:

- independently decide on data analysis methods, given a data set and a research question
- justify decision making when pre-processing messy data for data analysis.

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Learning goals and outline

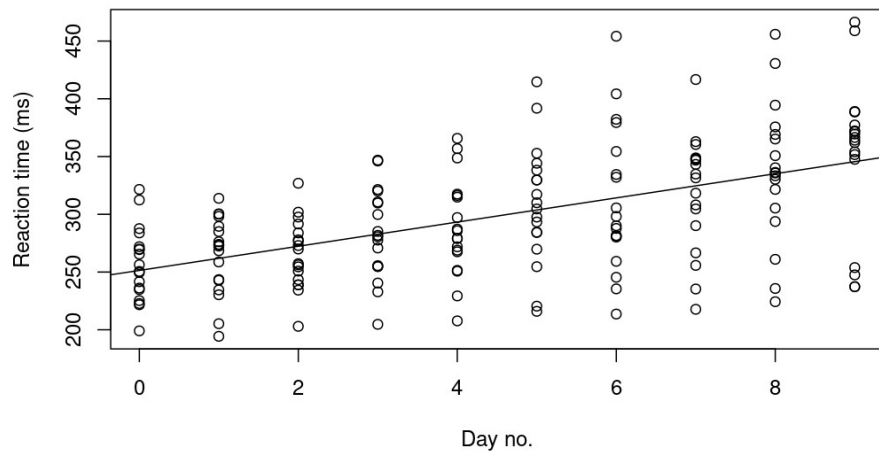
Multilevel linear regression

- 1) Understanding the motivations for doing multilevel modelling
 - Using all the data
 - Respecting the data distribution
- 2) Understanding the basics of multilevel modelling
 - Different kinds of design matrices, X and Z
 - Variance-covariance matrices
 - Pooling: complete pooling, no pooling and partial pooling

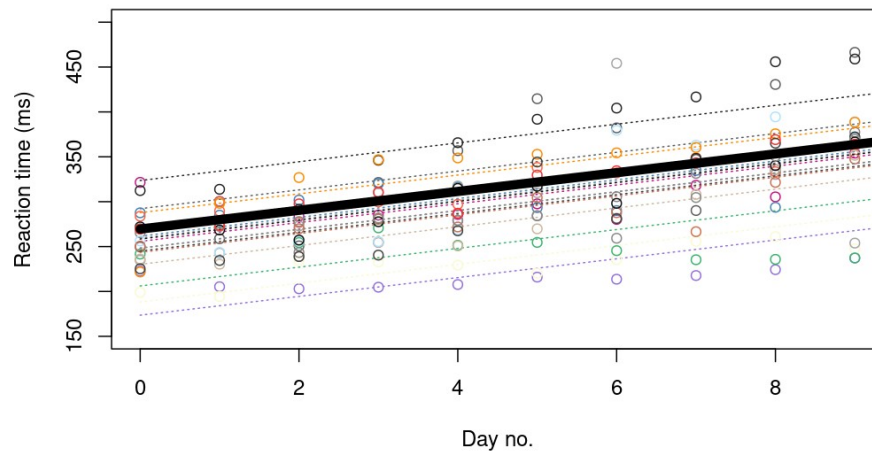
MAIN CONCEPTS

(Partial) Pooling

Pooled linear regression



Linear regression with subject-level intercepts



Design matrices

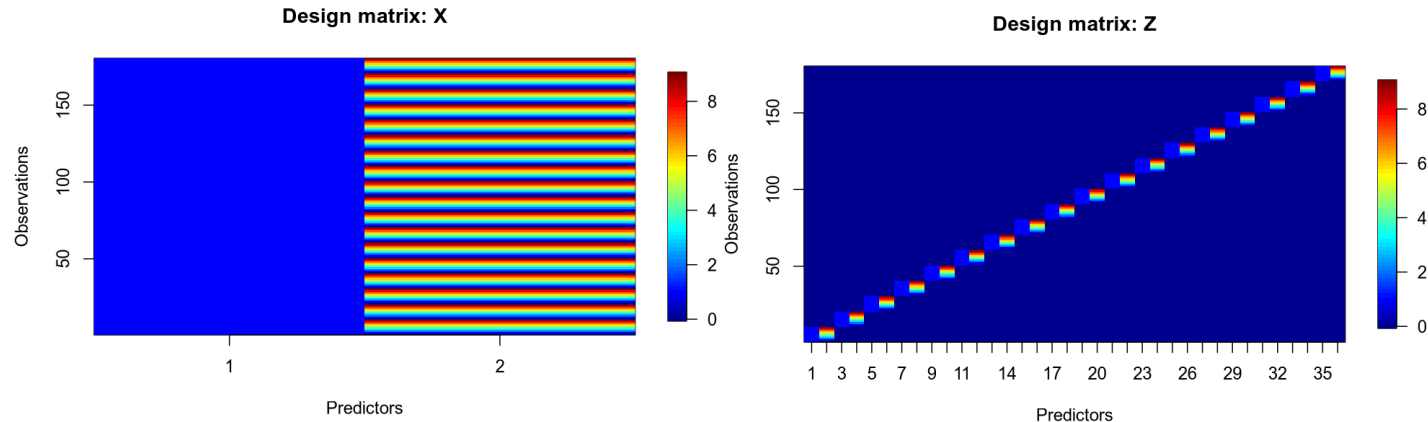
$$Y = X\beta + Zb + \epsilon$$

Level 1

Fixed
effects

Random
effects

Level 2



CONCEPTS THAT I SNUCK IN

How is shrinking done?

HOW IS **BIAS** ADDED?

u is the set of subject parameters

Penalised least squares:

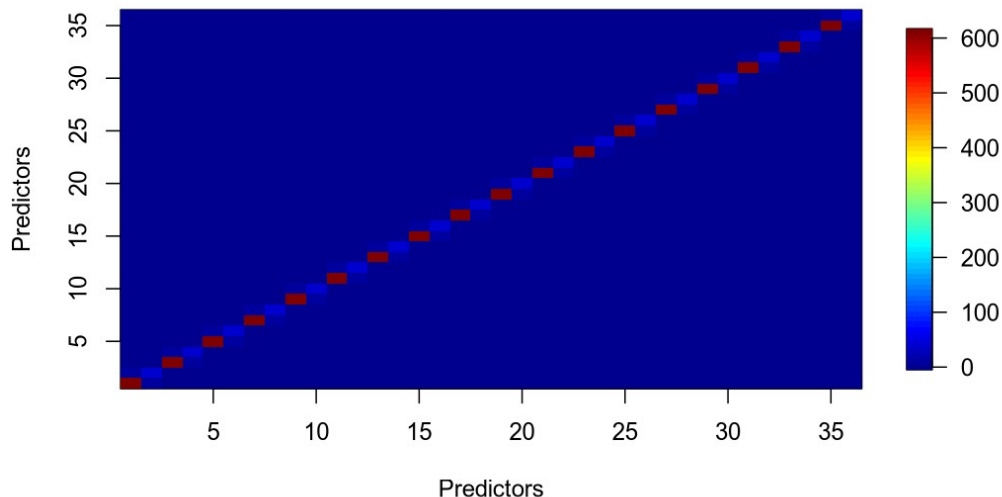
$$r^2(\theta, \beta, u) = \rho^2(\theta, \beta, u) + \|u\|^2 \quad (14)$$

$\rho^2(\theta, \beta, u)$ is the (weighted) residual sum of squares

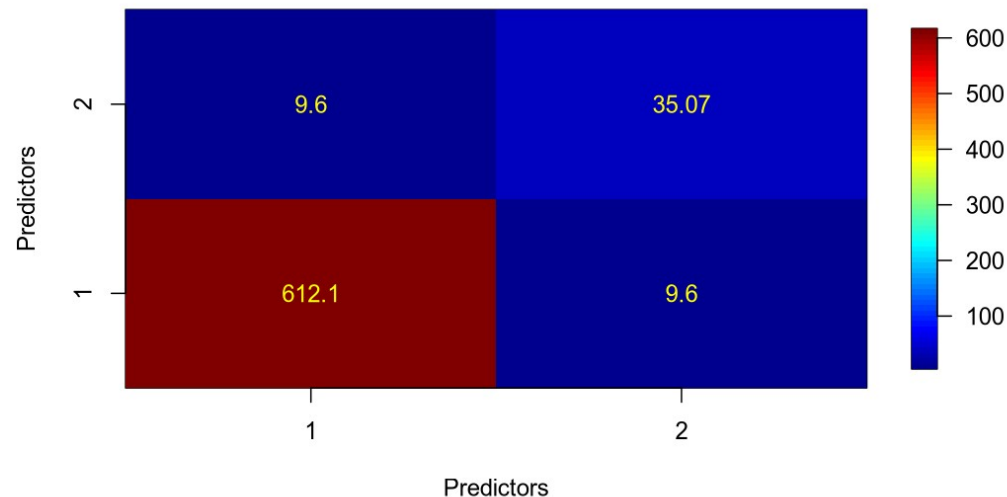
$\hat{\Sigma}_{\theta}$

Covariance

Variance-Covariance matrix



Variance-Covariance matrix (zoom)

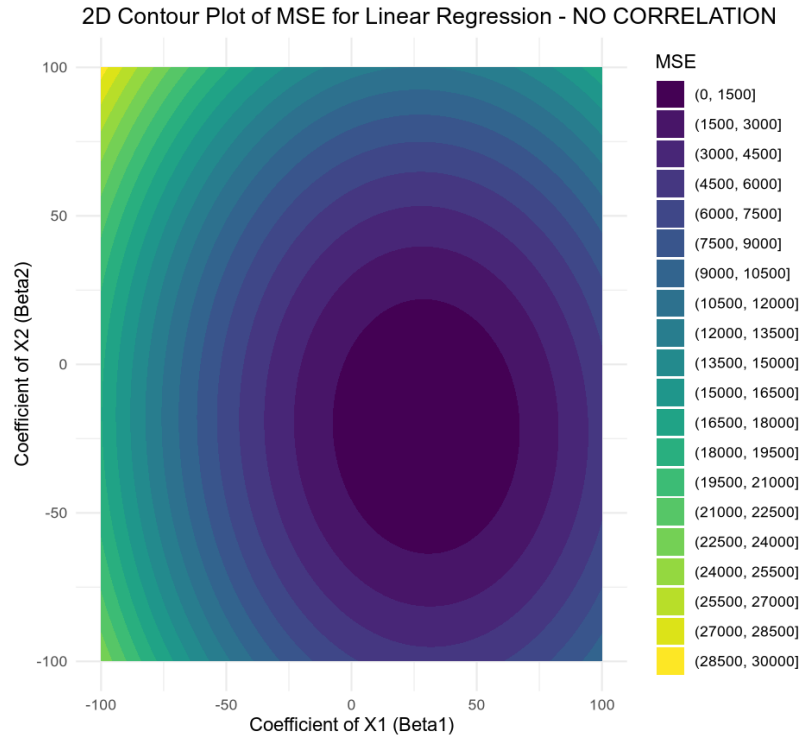


Q: What would the plot look like if there was no covariance between predictors?

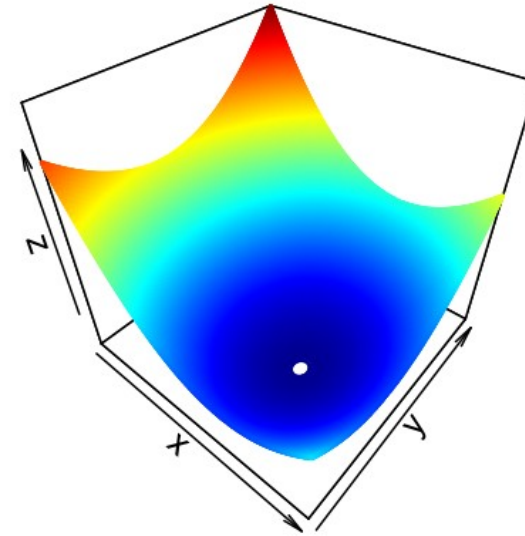
$$Corr = \frac{9.6}{\sqrt{(612.1 * 35.07)}} = 0.07$$

True function: $y = 31x_1 - 21x_2$
 $x_1 = \text{rnorm}(100); x_2 = \text{rnorm}(100)$

Gradient descent

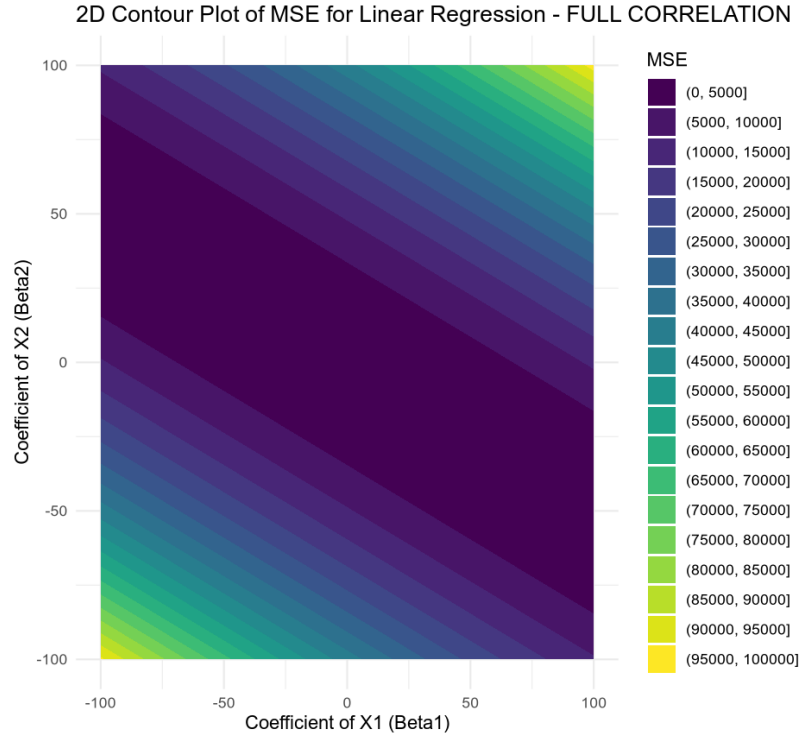


3D Contour Plot of MSE for Linear Regression - NO CORRELATION



Full rank

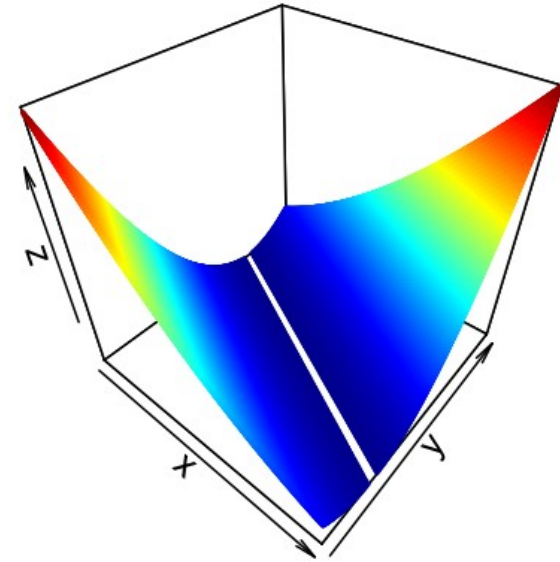
True function: $y = 31x_1 - 21x_2$
 $x_1 = rnorm(100); x_2 = 2x_1$



Singular

Gradient descent

3D Contour Plot of MSE for Linear Regression - FULL CORRELATION



The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Learning goals and outline

Link functions and fitting generalised linear multilevel models

- 1) Understanding that we can extend the scope of our general linear model by using appropriate link functions and data distributions
- 2) Understanding that we can extend this scope to multilevel modelling as well
- 3) Getting an understanding of maximum likelihood estimation

At least four ingredients needed

- 1) A data vector: $y = (y_1, \dots, y_n)$
- 2) Predictors: X and coefficients β , forming a linear predictor $X\beta$
- 3) A *link function* g : yielding a vector of transformed data $\hat{y} = g^{-1}(X\beta)$
that are used to model the data
- 4) A data distribution: $p(y | \hat{y})$

$$(X\beta = \beta_0 + X_1\beta_1 + \dots + X_k\beta_k)$$

Gelman A, Hill J (2006) Data Analysis Using Regression and Multilevel/Hierarchical Models. Cambridge University Press: Chapter 6

Logistic regression

$$g(x) = \log\left(\frac{x}{1-x}\right)$$

$p(y|\hat{y})$: the probability of y
when our fitted value is $\hat{y}$

$$\hat{y} = p(y=1)$$

$$p(y|\hat{y}) = \hat{y}^y (1-\hat{y})^{1-y}$$

$$y \in \{0, 1\}; \hat{y} \in [0, 1]$$

```
log.reg <- glm(am ~ wt, data=mtcars, family='binomial')
y <- log.reg$y
X <- model.matrix(log.reg)
beta.hat <- coef(log.reg)
lin.pred <- X %>% beta.hat
y.hat <- inv.g(lin.pred) # identical to fitted(log.reg)
likelihoods <- dbinom(y, size=1, prob=y.hat) # bernoulli
likelihood <- prod(likelihoods)
log.likelihood <- log(likelihood) ## check for yourself with logLik(log.reg)
res.deviance <- -2 * log.likelihood # null deviance is logLik(glm(am ~ 1))
n.predictors <- length(beta.hat)
AIC <- 2 * n.predictors + res.deviance
print(log.reg)
```

$$\text{PMF}_{\text{Bernoulli}} = p^k (1-p)^{1-k}$$

$$p \in [0, 1]; k \in \{0, 1\}$$

$$= \begin{cases} p & \text{if } k = 1, \\ 1 - p & \text{if } k = 0. \end{cases}$$

Poisson regression $g(x) = \log(x)$

$p(y|\hat{y})$: the probability of y
when our fitted value is $\hat{y}$

$$\hat{y} = \lambda$$

$$p(y|\hat{y}) = \frac{\hat{y}^y e^{-\hat{y}}}{y!}$$
$$y \in N_0; \hat{y} \in [0, \infty)$$

$$PMF_{\text{Poisson}} = \frac{\lambda^k e^{-\lambda}}{k!}$$

```
pois.reg <- glm(carb ~ wt, data=mtcars, family='poisson')
y <- pois.reg$y
X <- model.matrix(pois.reg)
beta.hat <- coef(pois.reg)
lin.pred <- X %*% beta.hat
y.hat <- inv.g(lin.pred) # identical to fitted(pois.reg)
likelihoods <- dpois(y, y.hat) # poisson
likelihood <- prod(likelihoods)
log.likelihood <- log(likelihood) ## check for yourself with logLik(pois.reg)
res.deviance <- 2 * sum(y * log(y/y.hat) - (y - y.hat))
n.predictors <- length(beta.hat)
AIC <- 2 * n.predictors - 2*log.likelihood
print(pois.reg)
```

Linear regression

$$g(x) = x$$

$p(y|\hat{y})$: the probability of y
when our fitted value is $\hat{y}$

$$\hat{y} = X\hat{\beta}$$

$$p(y|\hat{y}) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(\hat{y} - X\beta)^2}{2\sigma^2}} : \text{Gaussian distribution}$$

$$\sigma \in \mathbb{R}_{>0}$$

```
lin.reg      <- glm(mpg ~ wt, data=mtcars, family='gaussian')
y            <- lin.reg$y
n            <- length(y)
X            <- model.matrix(lin.reg)
beta.hat     <- coef(lin.reg)
lin.pred     <- X %*% beta.hat
y.hat        <- inv.g(lin.pred) # identical to fitted(lin.reg)
sigma.hat.squared <- 1/n * sum((y - y.hat)^2)
log.likelihood <- -n/2*log(2*pi) - n/2*log(sigma.hat.squared) - n/2
res.deviance  <- sum((y - y.hat)^2)
n.predictors  <- length(beta.hat) + 1 # sigma hat squared
AIC           <- 2 * n.predictors - 2*log.likelihood
print(lin.reg)
```

log-likelihood maximisation

$$l(\hat{\beta}, \hat{\sigma}^2 | y) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log(\hat{\sigma}^2) - \frac{n}{2}$$

$$l(\hat{\beta} | y) = \sum_{i=1}^n y_i \log(\hat{y}_i) + \log(1 - y_i)(1 - \hat{y}_i)$$

$$l(\hat{\beta} | y) = \sum_{i=1}^n y_i \log(\hat{y}_i) - \hat{y}_i - \log(y_i!)$$

Upshot:

With a link function g , and a data distribution $p(y|\hat{y})$, you can fit whatever you want

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Learning goals and outline

Evaluating Generalised linear mixed models

- 1) Learning tools for comparing models
 - 1) Variance explained
 - 2) Likelihood ratio tests
 - 3) Information criteria
- 2) Bridging to out-of-sample
 - 1) Introducing regularisation

R^2 – multilevel models

$$R_{GLMM}(m)^2 = \frac{\sigma_f^2}{\sigma_f^2 + \sigma_\alpha^2 + \sigma_\epsilon^2} \qquad R_{GLMM}(c)^2 = \frac{\sigma_f^2 + \sigma_\alpha^2}{\sigma_f^2 + \sigma_\alpha^2 + \sigma_\epsilon^2}$$

σ_f^2 : variance of the first-level effects

σ_α^2 : variance of the second-level effects

σ_ϵ^2 : unexplained variance

m : marginal variance

c : conditional variance

Likelihood-ratio test

$$LR = -2(l(\theta_2) - l(\theta_1))$$

```
print(anova(model.ranint, model.ranslope))
```

refitting model(s) with ML (instead of REML)

Data: sleepstudy

Models:

model.ranint: Reaction ~ Days + (1 | Subject)

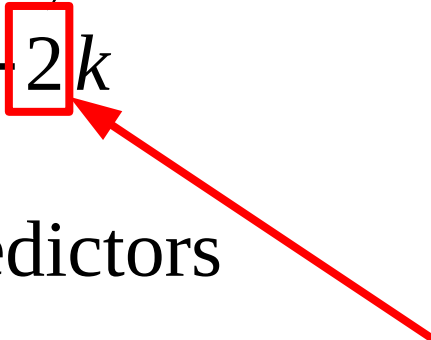
model.ranslope: Reaction ~ Days + (Days | Subject)

	npar	AIC	BIC	logLik	deviance	Chisq	Df	Pr(>Chisq)
model.ranint	4	1802.1	1814.8	-897.04	1794.1			
model.ranslope	6	1763.9	1783.1	-875.97	1751.9	42.139	2	7.072e-10 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Information criteria

$$\text{deviance} = -2l(\hat{\theta})$$

$$\text{AIC} = \text{deviance} + 2k$$


k : number of predictors

When we add k predictors that are pure noise, deviance is reduced by an amount corresponding to a χ^2 distribution with k degrees of freedom.

(Gelman and Hill, 2006)

“On average, a predictor needs to reduce the deviance by 2 in order to improve the fit to new data”

(Gelman and Hill, 2006, p. 525)

Evaluating multilevel models

- Our different ways of comparing models seem to rely on estimating the number of effective parameters, on which there is no clear consensus
- Collinearity is a problem for all methods we have covered
 - Collinearity leads to unstable models
 - What can we do about it?

Sneaking in **stability**, **regularisation** and **bias-variance decomposition**

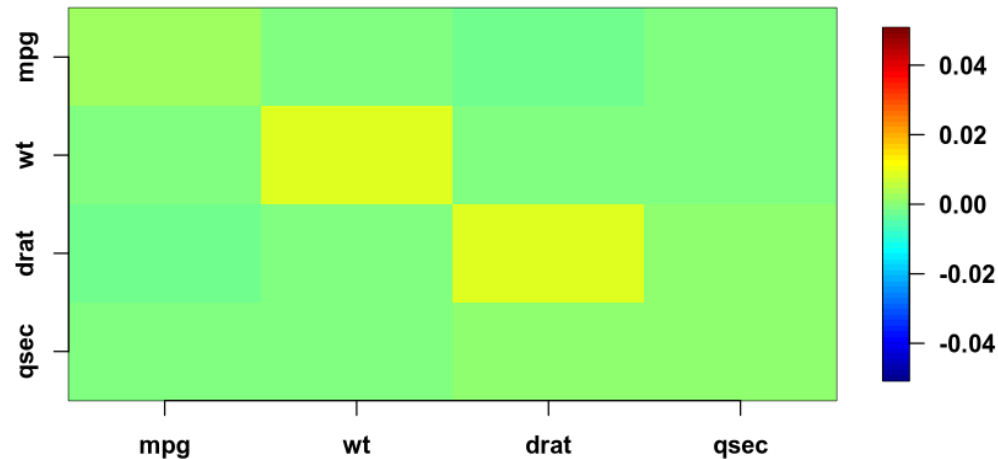
Stability of a model:

Feeding new data or adding new predictor variables will not change the parameter estimates a lot

Seems to be a good ingredient for a model that generalises well

Collinearity and regularisation

$$(X^T X + \lambda I)^{-1}; (\lambda = 100)$$



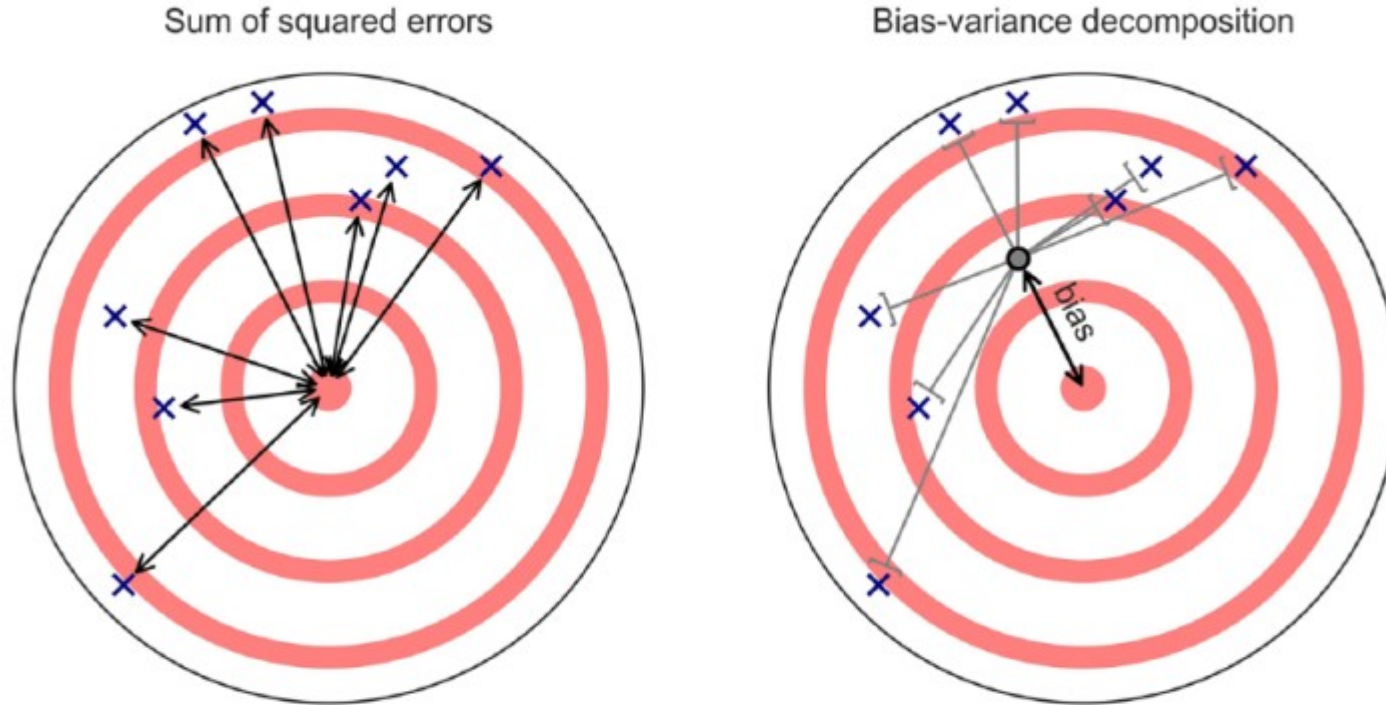


Fig. 3. Schematic illustration of the bias-variance decomposition. (Left) Under the classical error model, prediction error is defined as the sum of squared differences between true scores and observed scores (black lines). (Right) The bias-variance decomposition partitions the total sum of squared errors into two separate components: a bias term that captures a model's systematic tendency to deviate from the true scores in a predictable way (black line) and a variance term that represents the deviations of the individual observations from the model's expected prediction (gray lines).

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Learning goals and outline

Explanation and prediction

- 1) Understanding how error can be decomposed into bias and variance
- 2) Understanding when penalised regression may be helpful
- 3) Understanding how models can be evaluated by out-of-sample testing

Bias-variance decomposition

$$MSE = E[(f(x) - \hat{f}(x))^2] = bias(\hat{f}(x))^2 + var(\hat{f}(x)) + \sigma^2$$

$$bias(\hat{f}(x)) = E[\hat{f}(x)] - f(x)$$

MSE : mean squared error

$E[\]$: expected value (often the mean)

$f(x)$: true underlying function

$\hat{f}(x)$: estimated underlying function

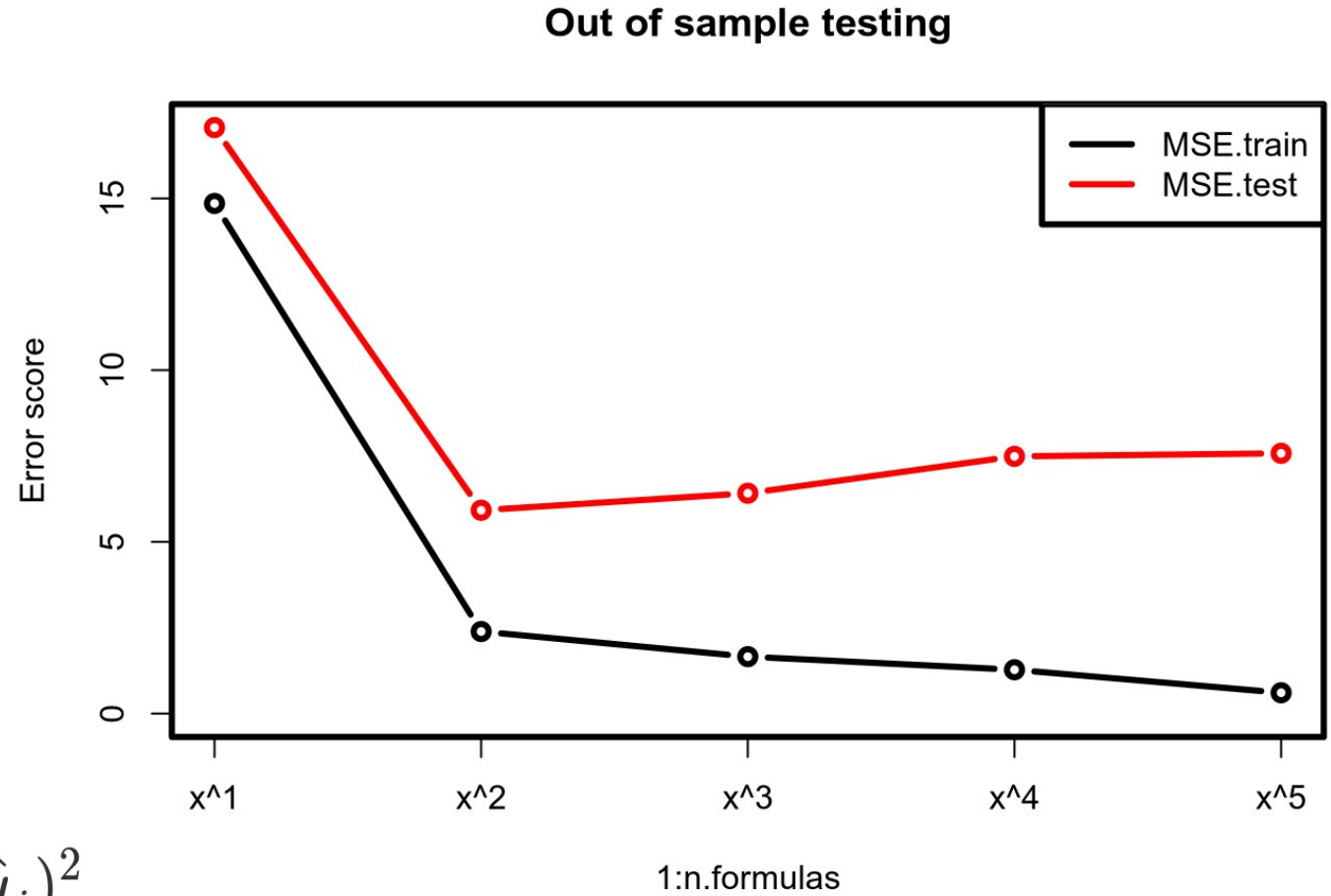
OUT OF SAMPLE TESTING as a way of evaluating models

Using mean squared error as a
measure of predictive power

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$\sigma=2$
 $n . obs=7$

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$



Bias-variance decomposition

So if greater **variance** is detrimental to prediction, we may be able improve prediction by introducing **bias** (and thereby reducing **variance**)

$$MSE = E[(f(x) - \hat{f}(x))^2] = bias(\hat{f}(x))^2 + var(\hat{f}(x)) + \sigma^2$$

L2 regularization, ridge

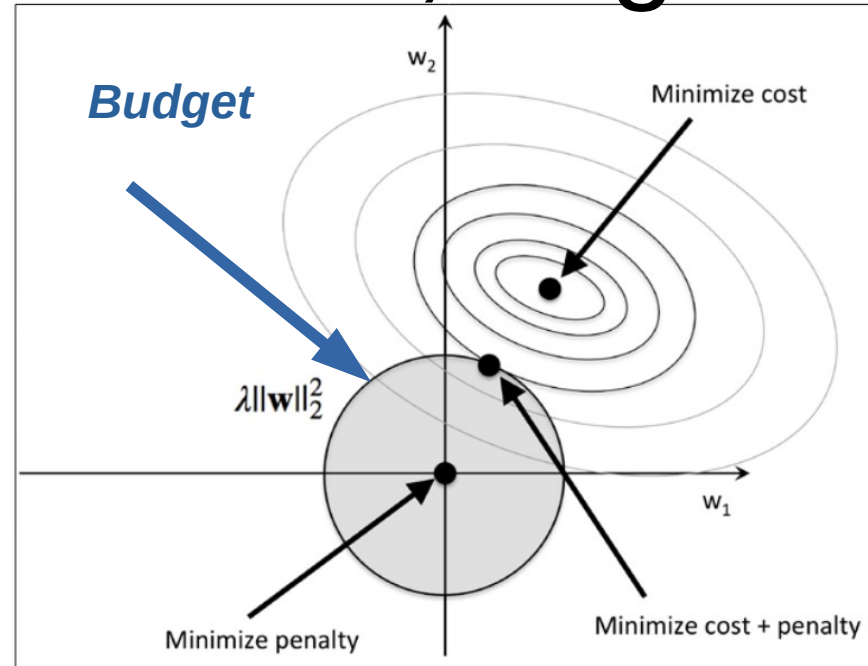
Why is the *budget* round?

Compare with a circle centred at (0,0)

$$x^2 + y^2 = r^2$$

$$w_1^2 + w_2^2 = r^2$$

$$\|w\|_2 = \sqrt{(w_1^2 + w_2^2)}$$



a
penalising
term
adding
bias
(remember
partial
pooling)

(p. 114: Raschka, 2015)

$$J(w)_{Ridge} = \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2 + \lambda \|w\|_2^2$$

L1 regularization, lasso

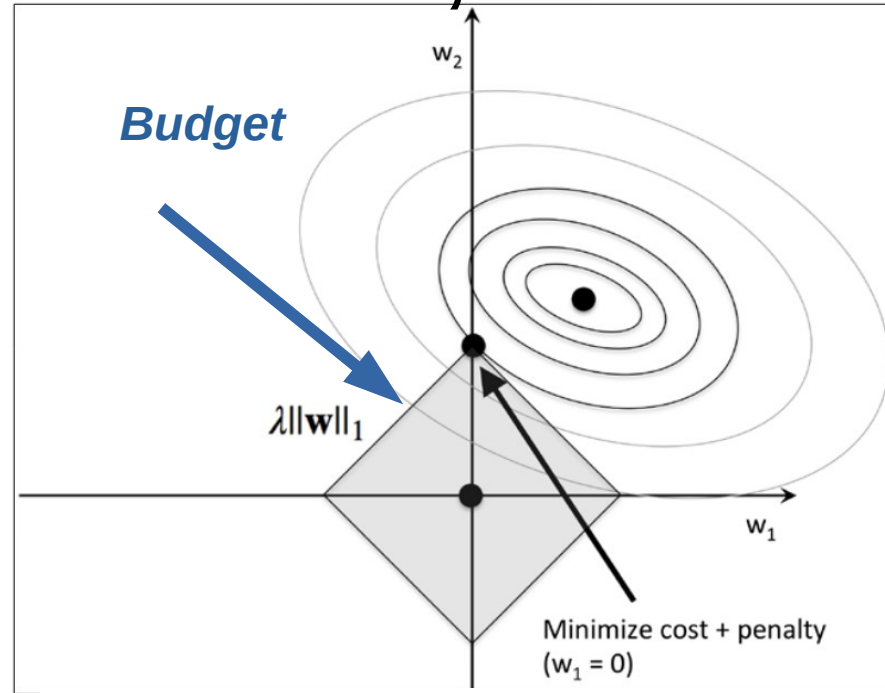
Why is the *budget* square?

$$\|w\|_1 = |w_1| + |w_2|$$

if $w_1 = \max(w_1)$ then $w_2 = 0$

if $w_2 = \max(w_2)$ then $w_1 = 0$

(p. 115: Raschka, 2015)



a
penalising
term
adding
bias
(remember
partial
pooling)

$$J(w)_{LASSO} = \sum_{i=1}^n \left(y^{(i)} - \hat{y}^{(i)} \right)^2 + \lambda \|w\|_1$$

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

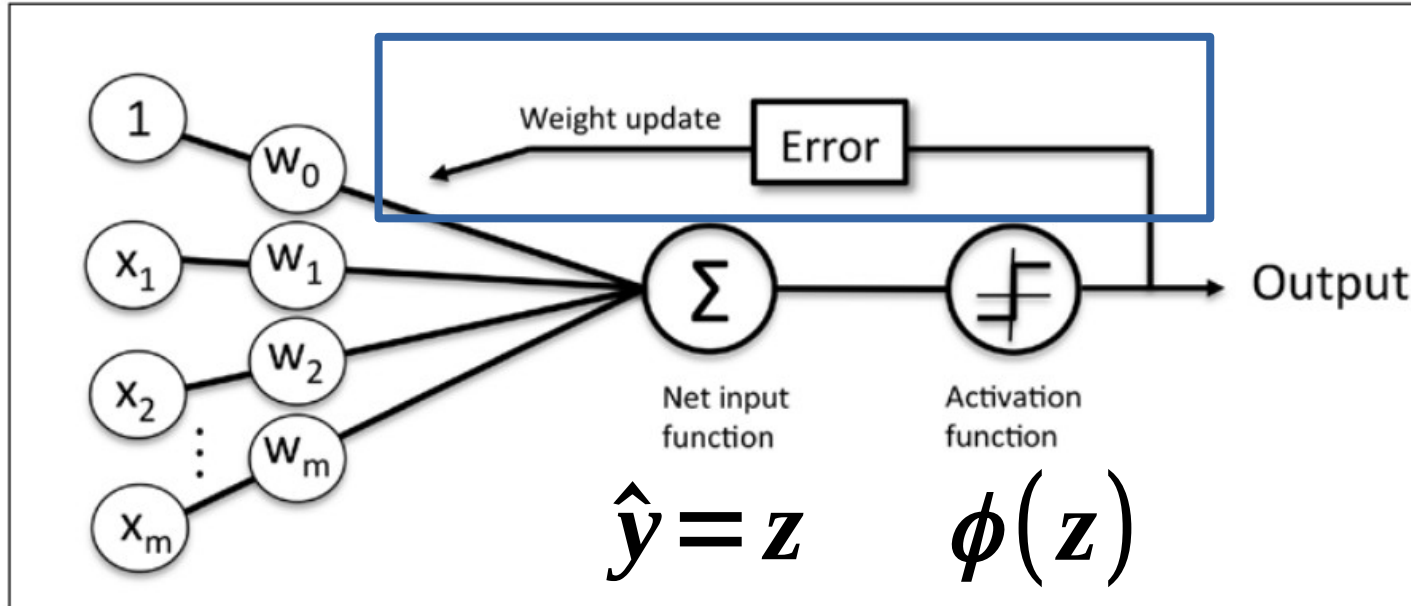
Learning goals and outline

Mid-way evaluation and Machine Learning Intro

- 1) Learning some early *classification* methods
 - Perceptron and ADALine
 - Classification depends on having a quantiser function
- 2) Learning how linear *regression* (with biasing penalties) can be constructed and cross-validated

We are trying to eliminate or minimise
classification errors

Perceptron: Graphical summary



(p. 24: Raschka, 2015)

Compute the output value $\hat{y}$



```
def net_input(self, X):  
    """Calculate net input"""  
    return np.dot(X, self.w_[1:]) + self.w_[0]  
  
def predict(self, X):  
    """Return class label after unit step"""  
    return np.where(self.net_input(X) >= 0.0, 1, -1)
```



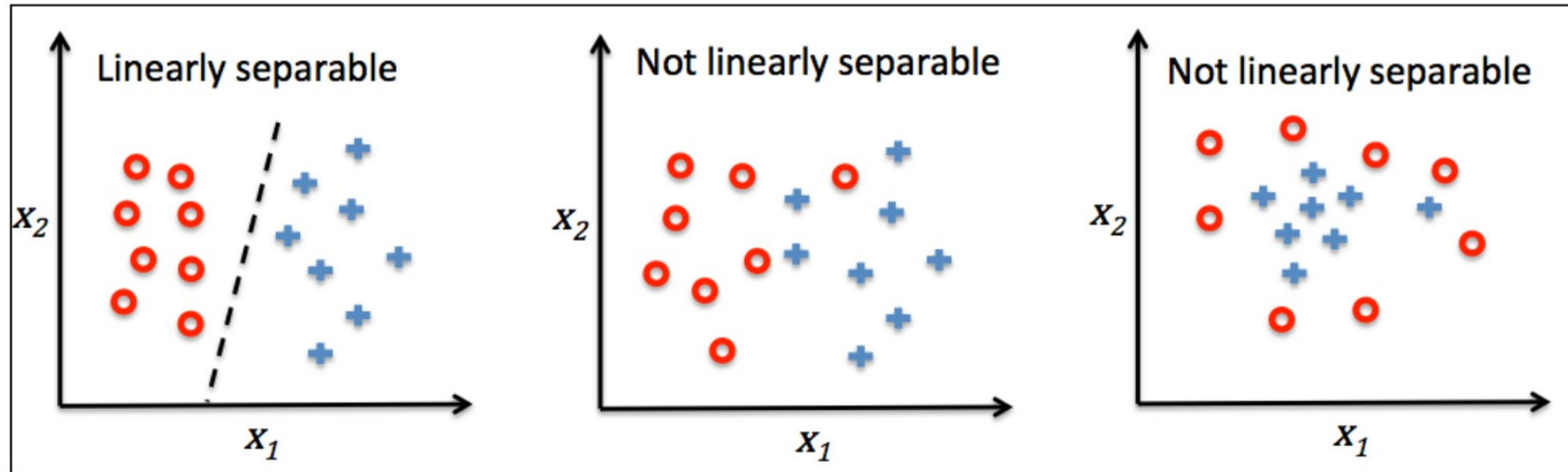
$$\phi(z) = \begin{cases} 1 & \text{if } z \geq \theta \\ -1 & \text{otherwise} \end{cases}$$

Specifying methods of *Perceptron*

1. Initialize the weights to 0 or small random numbers.
2. For each training sample $\mathbf{x}^{(i)}$ perform the following steps:
 1. Compute the output value $\hat{y}$.
 2. Update the weights.

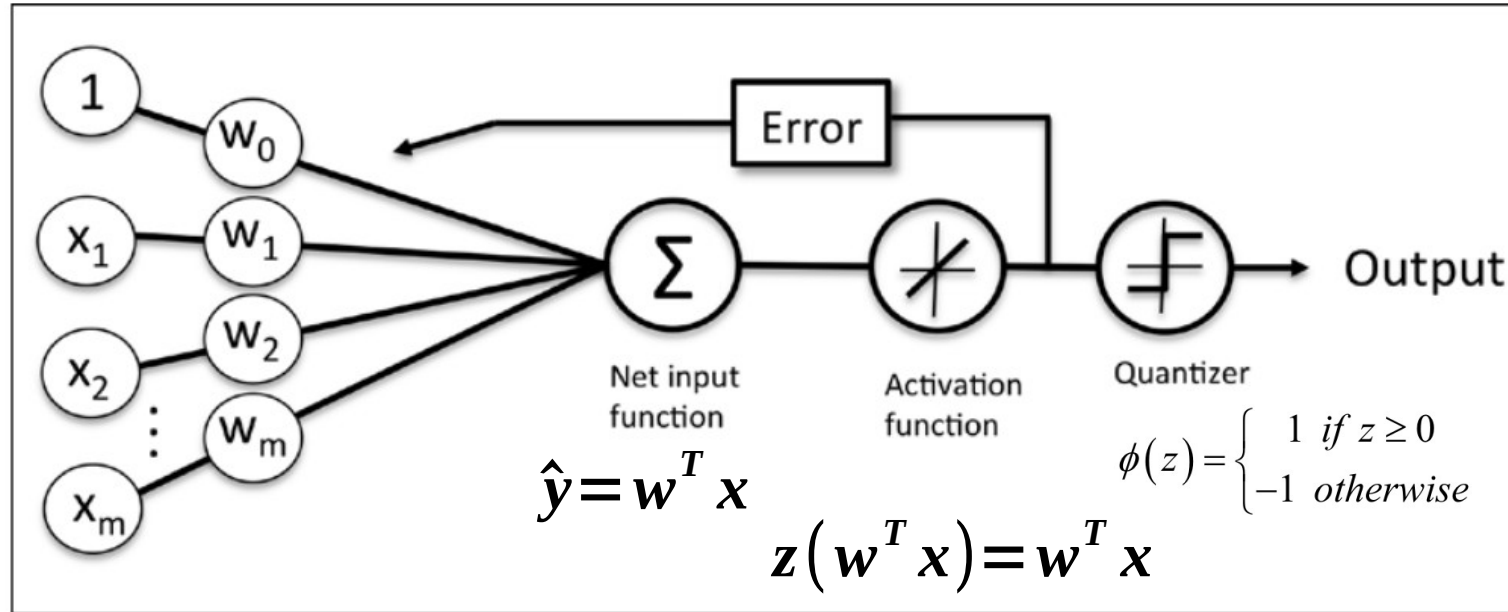
```
def fit(self, X, y):  
    """ Fit training data.  
  
    Parameters  
    -----  
    X : {array-like}, shape = [n_samples, n_features]  
        Training vectors, where n_samples  
        is the number of samples and  
        n_features is the number of features.  
    y : array-like, shape = [n_samples]  
        Target values.  
  
    Returns  
    -----  
    self : object  
  
    """  
    self.w_ = np.zeros(1 + X.shape[1])  
    self.errors_ = []  
  
    for _ in range(self.n_iter):  
        errors = 0  
        for xi, target in zip(X, y):  
            update = self.eta * (target - self.predict(xi))  
            self.w_[1:] += update * xi  
            self.w_[0] += update  
            errors += int(update != 0.0)  
            self.errors_.append(errors)  
        return self
```


Convergence only possible when linearly separable



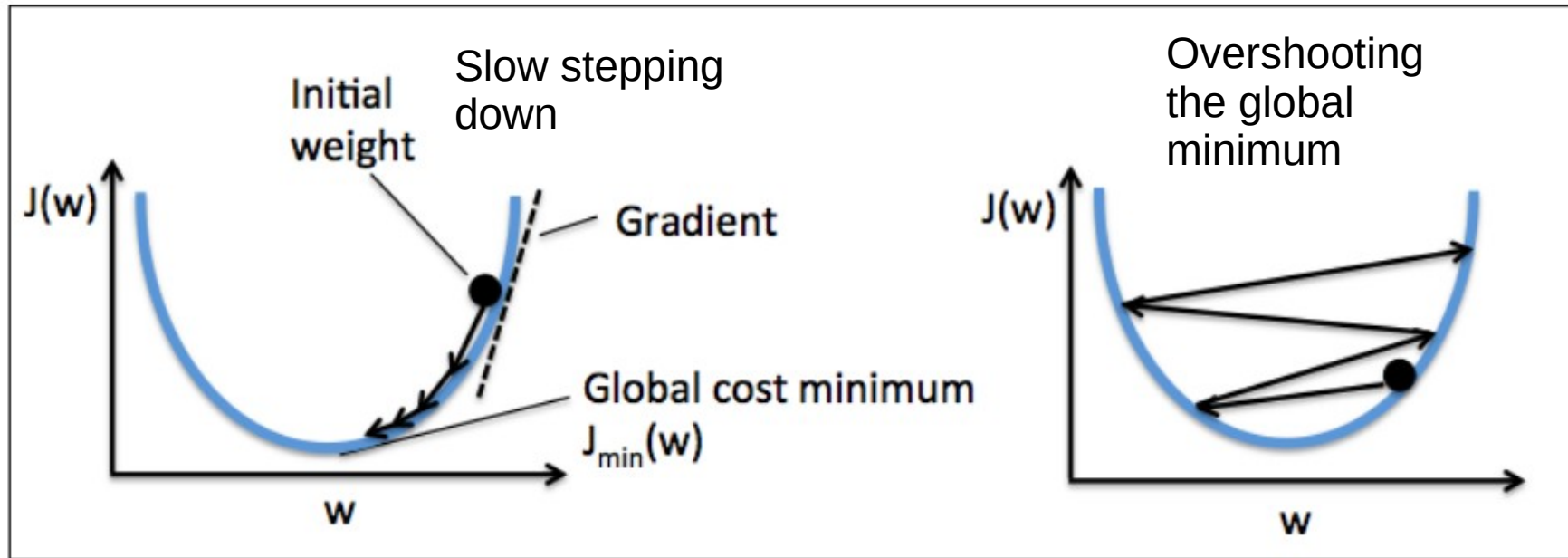
(p. 23: Raschka, 2015)

ADaptive Linear NEuron (ADALINE)



$$w^T x = w_0 x_0 + w_1 x_1 + \dots + w_{m-1} x_{m-1} + w_m x_m$$

Gradient descent



$$J(w) = (\sum (y - \hat{y})^2) / 2$$

(p. 40: Raschka, 2015)

Methods

$$\hat{y} = w^T x$$

$$z(w^T x) = w^T x$$

$$\phi(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ -1 & \text{otherwise} \end{cases}$$

```
def net_input(self, X):  
    """Calculate net input"""  
    return np.dot(X, self.w_[1:]) + self.w_[0]  
  
def activation(self, X):  
    """Computer linear activation"""  
    return self.net_input(X)  
  
def predict(self, X):  
    """Return class label after unit step"""  
    return np.where(self.activation(X) >= 0.0, 1, -1)
```

The *fit* method

eta (η): learning rate (a constant)

output: $X\hat{\beta} = \hat{y}$

errors: $y - \hat{y}$

X.T.dot(errors): $X^T(y - \hat{y})$: steepest gradient

$\eta X^T(y - \hat{y}) = \Delta w_1 + \Delta w_2 + \dots + \Delta w_{m-1} + \Delta w_m$

cost function: $(\sum (y - \hat{y})^2) / 2$

```
def fit(self, X, y):
    """ Fit training data.

    Parameters
    -----
    X : {array-like}, shape = [n_samples, n_features]
        Training vectors, where n_samples
        is the number of samples and
        n_features is the number of features.
    y : array-like, shape = [n_samples]
        Target values.

    Returns
    -----
    self : object

    """
    self.w_ = np.zeros(1 + X.shape[1])
    self.cost_ = []

    for i in range(self.n_iter):
        output = self.net_input(X)
        errors = (y - output)
        self.w_[1:] += self.eta * X.T.dot(errors)
        self.w_[0] += self.eta * errors.sum()
        cost = (errors**2).sum() / 2.0
        self.cost_.append(cost)
    return self
```

Interim summary

- You have learnt to fit multilevel models, where you **bias** towards the group mean
- You have learnt to fit non-normal data by using link functions, where you maximise log-likelihood (similar to minimising error with gradient descent)
- You have learnt to evaluate those models using classical tools like R^2 , log-likelihood and AIC
- You have learnt that out-of-sample generalisation may provide a better check on the model's stability
- You have learnt that to generalise well, you sometimes need to add **bias** to prevent overfitting

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

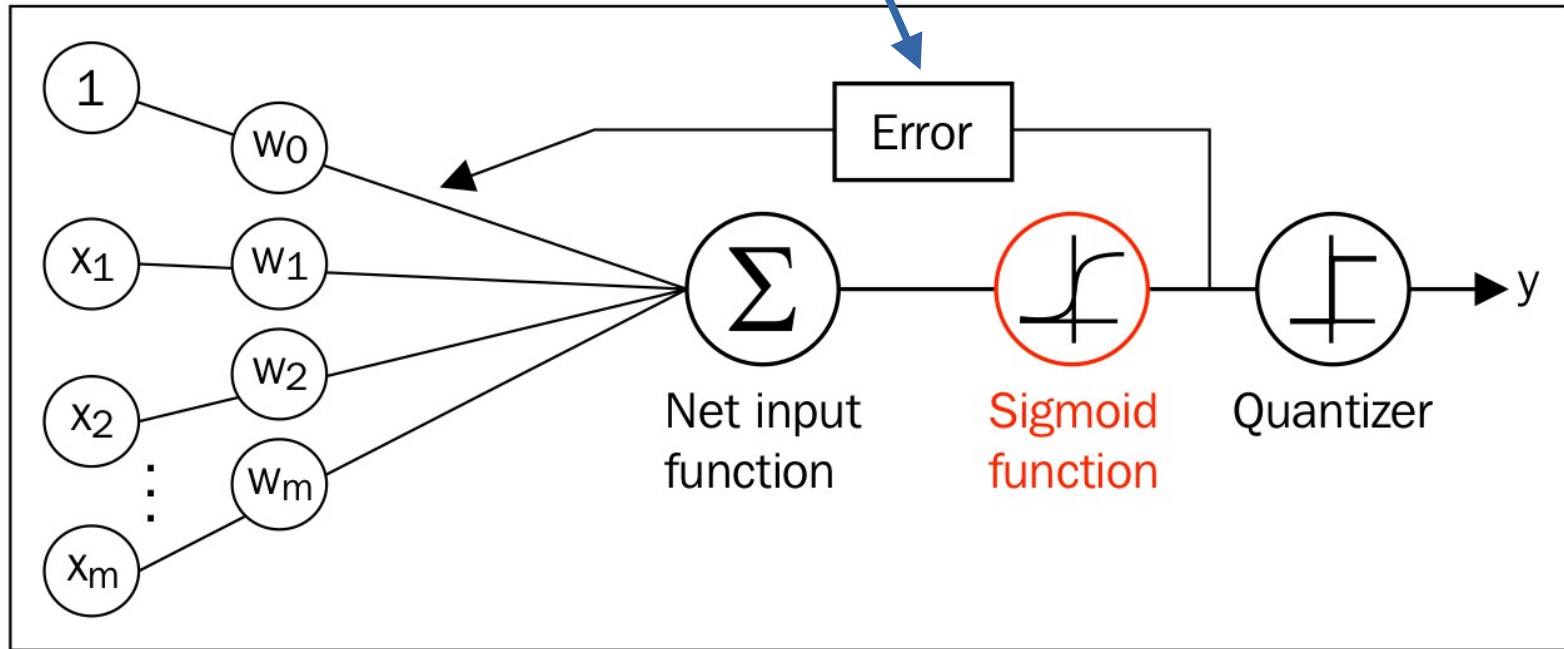
Learning goals and outline

Linear and logistic regression revisited (machine learning)

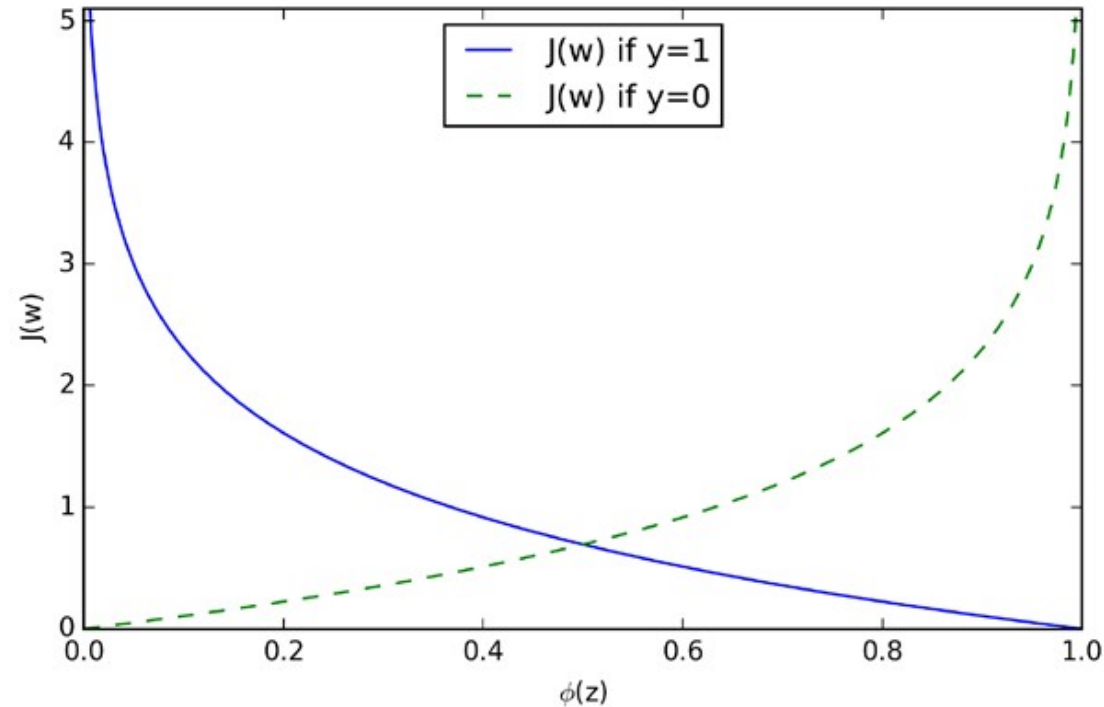
- 1) Understanding how linear and logistic regression can be adapted to a classification framework
- 2) Understanding how gradient descent can provide a common framework across regression fits

What's our cost function, $J(w)$?

i.e. how do we calculate this?



Cost function for one instance



Machine learning and link functions

- Gradient descent can be put into a common framework

- $\Delta w_j = -\eta \frac{\delta J}{\delta w_j} = \eta \sum_{i=1}^n (\mathbf{y}_i - \phi(\mathbf{z}_i)) x_j$

- we would thus also be able to do Poisson machine learning (not covered here), and any other transformation that would be meaningful
- Now, we need to move beyond fitting and test

- Standardisation

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x}$$

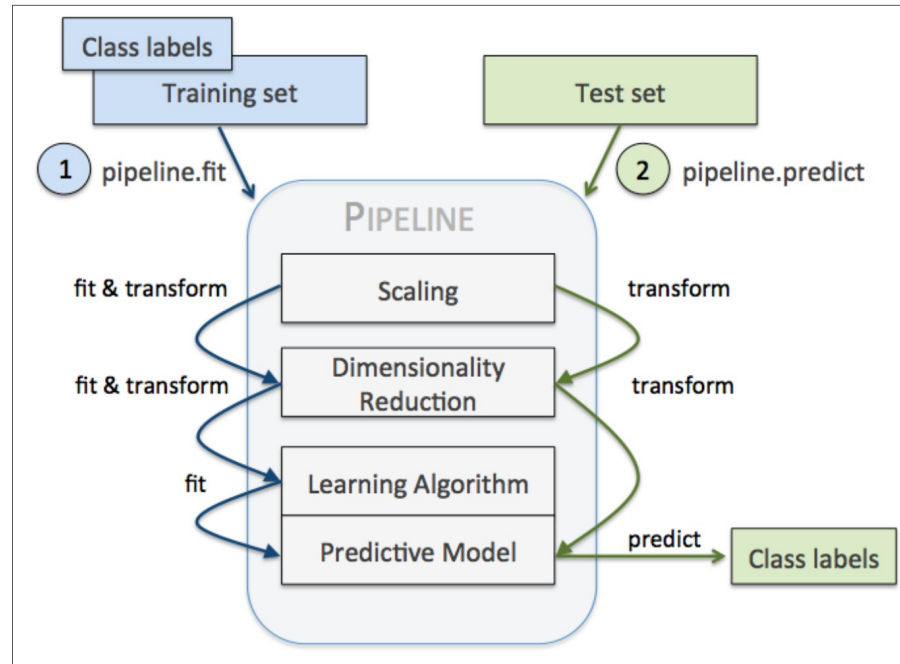
μ_x : sample mean for the feature: x

σ_x : sample standard deviation for the feature: x

Brings data onto a normal distribution with $\mu=0$ and $\sigma=1$



General structure



(p. 172: Raschka, 2015)

Can be implemented in *scikit-learn* (in four weeks)

```
from sklearn.pipeline import Pipeline  
  
pipe = Pipeline(steps=[("scaler", scaler), ("pca", pca), ("logistic", logistic)])
```

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

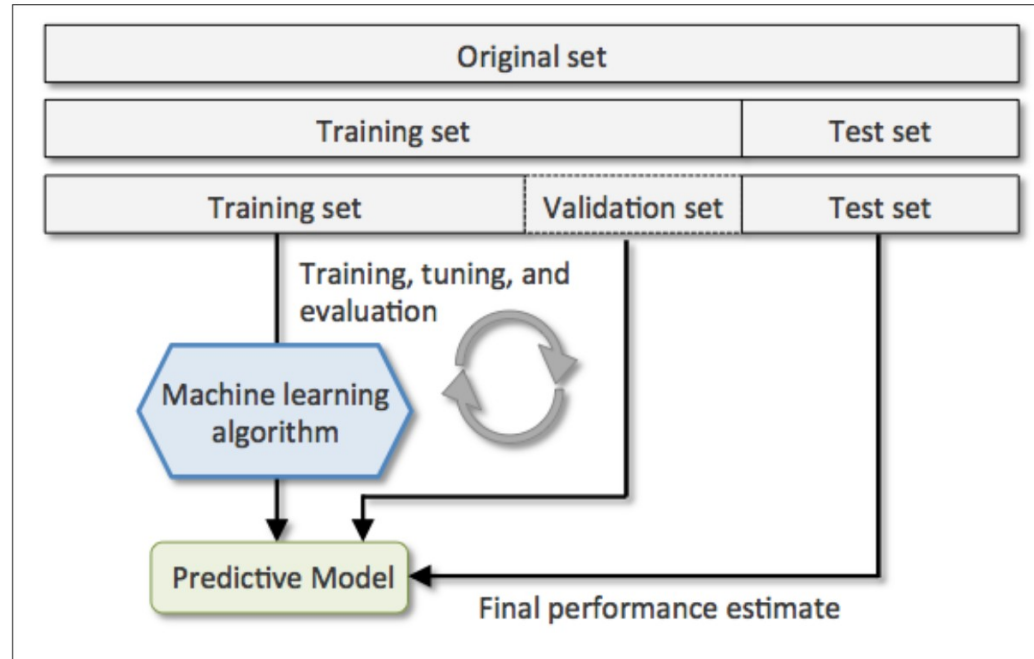
Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Learning goals and outline

Model evaluation and hyperparameter tuning

- 1) Understanding how cross-validation can improve validity
- 2) Understanding and being able to apply *Pipeline* from *scikit-learn*
- 3) Being able to run an analysis optimising the hyperparameters

Validation set



(p. 174: Raschka, 2015)

K-fold cross validation



(p. 176: Raschka, 2015)

Grid Search

```
##%% GRID SEARCH CV - setup

from sklearn.model_selection import GridSearchCV
param_range_C = [1e-4, 1e-3, 1e-2, 1e-1, 1e0, 1e1, 1e2, 1e3, 1e4]
param_range_PCA = np.arange(1, 15)
## Dictionary entry: name, e.g. 'clf' what it was called in the pipeline,
## two underscores and then the name of the parameter, e.g. C or n_components
param_grid = [{'clf__C': param_range_C, # you need two underscores
               'pca__n_components': param_range_PCA}]

grid_search = GridSearchCV(estimator=pipe_lr, param_grid=param_grid,
                           cv=10, n_jobs=-1) ## will use StratifiedKfold
```

```
##%% GRID SEARCH CV - fit

grid_search.fit(X, y)
print(grid_search.best_params_)
```

```
{'clf__C': 1.0, 'pca__n_components': 9}
```

Randomised grid search



Although grid search is a powerful approach for finding the optimal set of parameters, the evaluation of all possible parameter combinations is also computationally very expensive. An alternative approach to sampling different parameter combinations using scikit-learn is randomized search. Using the `RandomizedSearchCV` class in scikit-learn, we can draw random parameter combinations from sampling distributions with a specified budget. More details and examples for its usage can be found at http://scikit-learn.org/stable/modules/grid_search.html#randomized-parameter-optimization.

```
# specify parameters and distributions to sample from
param_dist = {
    "average": [True, False],
    "l1_ratio": stats.uniform(0, 1),
    "alpha": stats.loguniform(1e-2, 1e0),
```

https://scikit-learn.org/stable/auto_examples/model_selection/plot_randomized_search.html#sphx-glr-auto-examples-model-selection-plot-randomized-search-py

Hyperparameters and Pipelines

- The space of hyperparameters can be quite large
 - Therefore we need consistent ways of exploring them
- Using a Pipeline is a good way to control things at a conceptual level within getting lost in the nitty-gritty
 - But do visit the nitty-gritty every now and then to make sure you understand what is going on

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

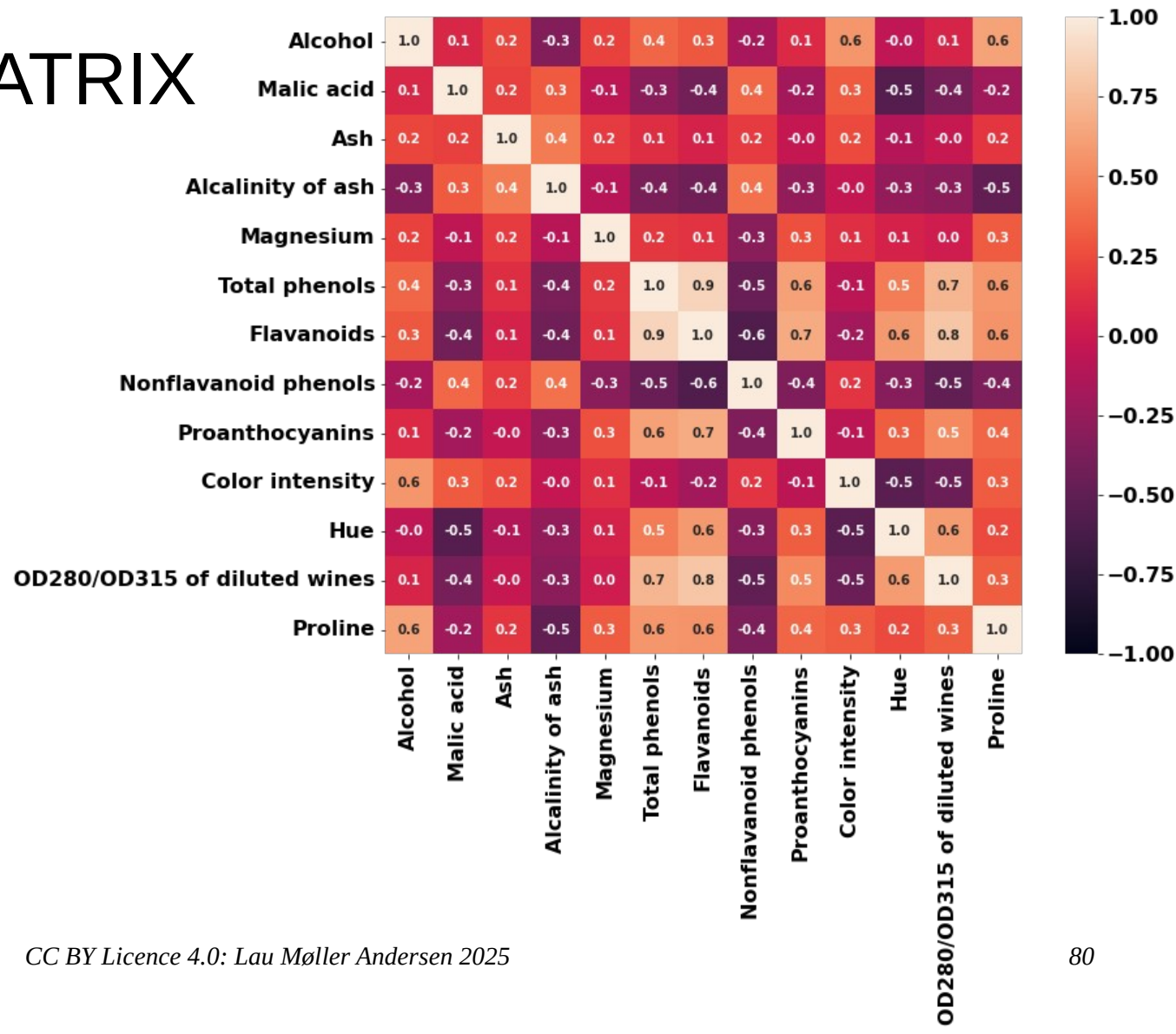
Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Learning goals and outline

Dimensionality Reduction, Principled Component Analysis (PCA)

- 1) Learning how we can extract the features that explain the most variance
- 2) Understanding how that can improve classification
- 3) Get acquainted with the concept of a eigenvector

COVARIANCE MATRIX



After PCA

```
## plotting eigenvalue plots

from sklearn.decomposition import PCA

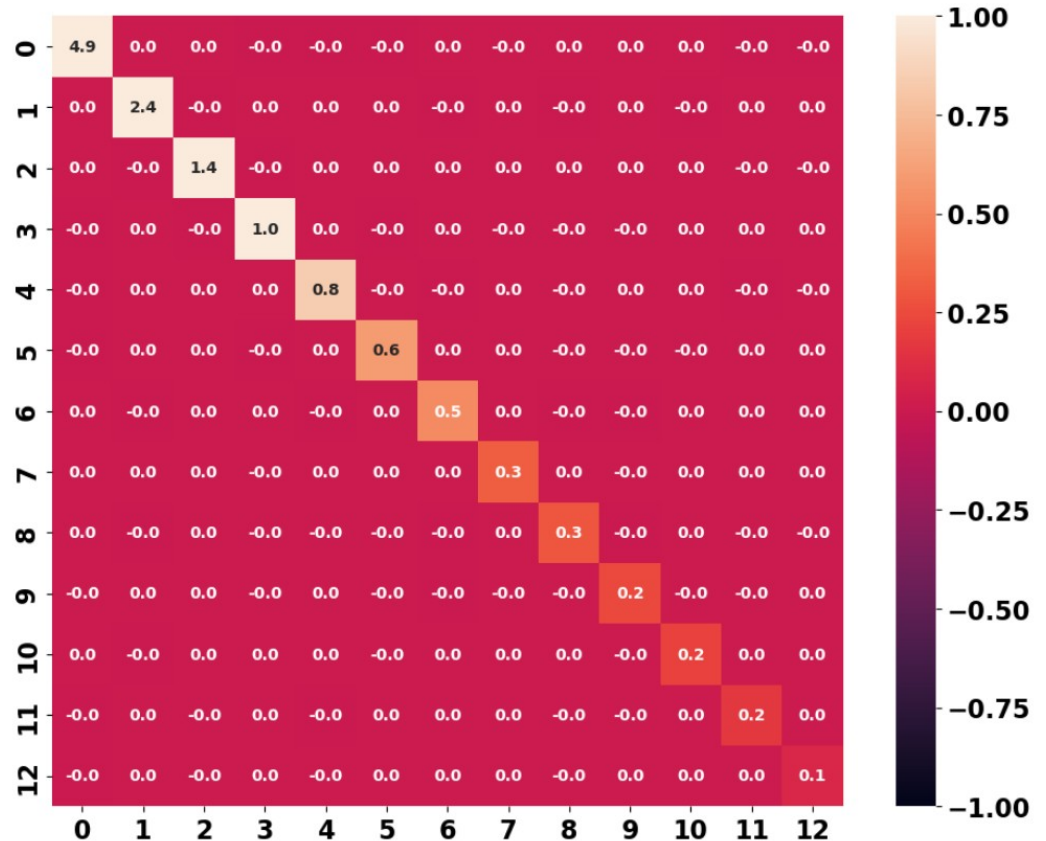
pca = PCA(n_components=X_train.shape[1])

X_train_std_pca = pca.fit_transform(X_train_std)

n = X_train.shape[0]

cov_pca = 1/n * X_train_std_pca.T @ X_train_std_pca

plt.figure(figsize=(12, 9))
sns.heatmap(cov_pca, cbar=True, annot=True, square=True,
            fmt='.1f',
            vmin=-1.0, vmax=1.0)
```



Derivation of optimisee

unit vector: $u^T u = 1$

covariance matrix: $C = \frac{1}{n} \sum_{i=1}^n x_i x_i^T$

maximise variance retained: $\max \sum_{i=1}^n (x_i^T u)^2$

write out the square: $\max \sum_{i=1}^n x_i^T u x_i^T u$

change the order of the first two factors: $\max \sum_{i=1}^n u^T x_i x_i^T u$

move u outside: $\max u^T \sum_{i=1}^n x_i x_i^T u$

insert covariance C : $\max u^T C u$

Lagrange multipliers

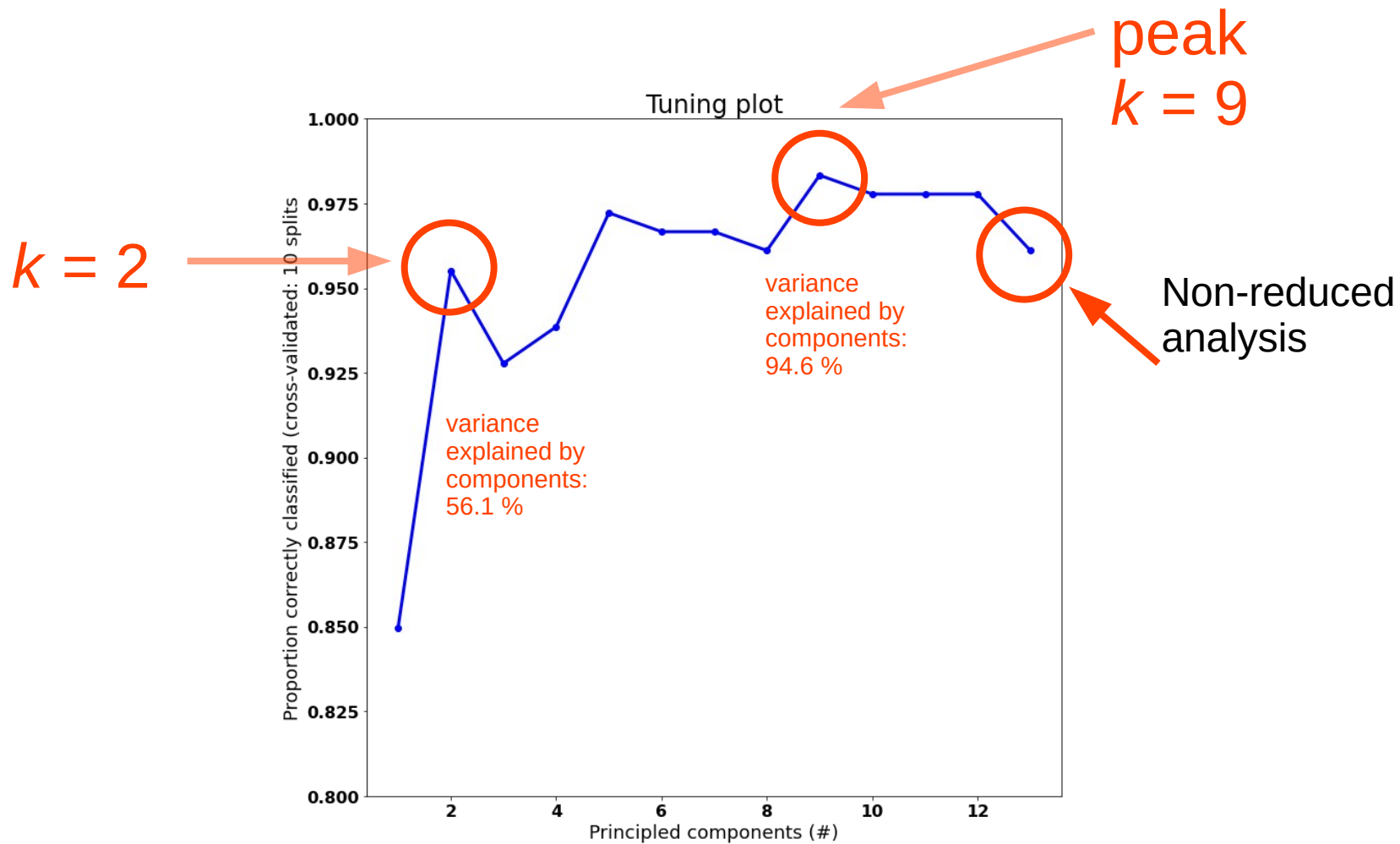
- Useful in optimisation problems with constraints
 - optimisee: $\max \mathbf{u}^T \mathbf{C} \mathbf{u}$
 - constraint: $\mathbf{u}^T \mathbf{u} = 1$
- Subtract the constraint equation set equal to 0 (multiplied by a constant, λ) from the optimisee
- Then, find the derivative for $\mathbf{u}$ (set it to zero (find the extremum))
- And *lo and behold*, the $\mathbf{u}$ that maximises the optimisee is something you know from Methods 2 – namely the eigenvectors

$$L(\mathbf{u}) = \mathbf{u}^T \mathbf{C} \mathbf{u} - \lambda (\mathbf{u}^T \mathbf{u} - 1)$$

$$\nabla(\mathbf{u}^T \mathbf{C} \mathbf{u}) - \lambda \nabla(\mathbf{u}^T \mathbf{u} - 1) = 0$$

$$2\mathbf{C} \mathbf{u} - 2\lambda \mathbf{u} = 0$$

$$\mathbf{C} \mathbf{u} = \lambda \mathbf{u}$$



The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Learning goals and outline

Neural Networks

- 1) Understanding the idea of a multilayer network
- 2) Understanding that it is partly a string of operations that we already know how to perform, e.g. logistic regression or linear regression
- 3) Getting the beginning of an understanding of backpropagation and how it can improve gradient descent

Three non-mysterious steps

- 1) Starting at the input layer, we **forward propagate** the patterns of the training data through the network to generate an output
- 2) Based on the network's output, we calculate the error that we want to minimise using a **cost function**
- 3) We **backpropagate** the error, find its derivative with respect to each weight in the network, and update the model

(p. 347: Raschka, 2015)

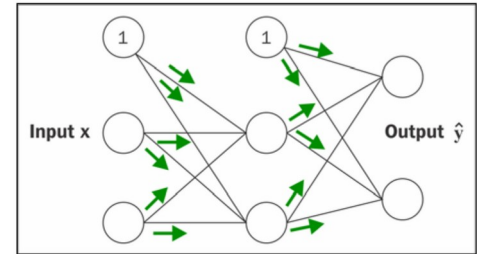
Feedforward steps summarised

$$\mathbf{Z}^{(2)} = \mathbf{W}^{(1)} \left[\mathbf{A}^{(1)} \right]^T \quad (\text{net input of the hidden layer})$$

$$\mathbf{A}^{(2)} = \phi\left(\mathbf{Z}^{(2)}\right) \quad (\text{activation of the hidden layer})$$

$$\mathbf{Z}^{(3)} = \mathbf{W}^{(2)} \mathbf{A}^{(2)} \quad (\text{net input of the output layer})$$

$$\mathbf{A}^{(3)} = \phi\left(\mathbf{Z}^{(3)}\right) \quad (\text{activation of the output layer})$$



Cost function

LOGISTIC REGRESSION

```
def _get_cost(self, y_enc, output, w1, w2):
    """Compute cost function.

    Parameters
    -----
    y_enc : array, shape = (n_labels, n_samples)
        one-hot encoded class labels.
    output : array, shape = [n_output_units, n_samples]
        Activation of the output layer (feedforward)
    w1 : array, shape = [n_hidden_units, n_features]
        Weight matrix for input layer -> hidden layer.
    w2 : array, shape = [n_output_units, n_hidden_units]
        Weight matrix for hidden layer -> output layer.

    Returns
    -----
    cost : float
        Regularized cost.

    """
    term1 = -y_enc * (np.log(output))
    term2 = (1.0 - y_enc) * np.log(1.0 - output)
    cost = np.sum(term1 - term2)
    L1_term = self._L1_reg(self.l1, w1, w2)
    L2_term = self._L2_reg(self.l2, w1, w2)
    cost = cost + L1_term + L2_term
    return cost
```

```
def _L1_reg(self, lambda_, w1, w2):
    """Compute L1-regularization cost"""
    return (lambda_/2.0) * (np.abs(w1[:, 1:]).sum() +
                             np.abs(w2[:, 1:]).sum())
```

```
def _L2_reg(self, lambda_, w1, w2):
    """Compute L2-regularization cost"""
    return (lambda_/2.0) * (np.sum(w1[:, 1:] ** 2) +
                             np.sum(w2[:, 1:] ** 2))
```

$$l(\hat{\beta}|y) = \sum_{i=1}^n y_i \log(\hat{y}_i) + \log(1 - y_i)(1 - \hat{y}_i)$$

Backpropagation

WE NEED TO MINIMISE $J(\mathbf{w})$ FOR EACH WEIGHT, w

$$\frac{\partial}{\partial w_{j,i}^{(l)}} J(\mathbf{w}) = a_j^{(l)} \delta_i^{(l+1)}$$

error from next layer

activation
from current
layer

l : n_layer
 j : n_activation_unit
 i : n_sample

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

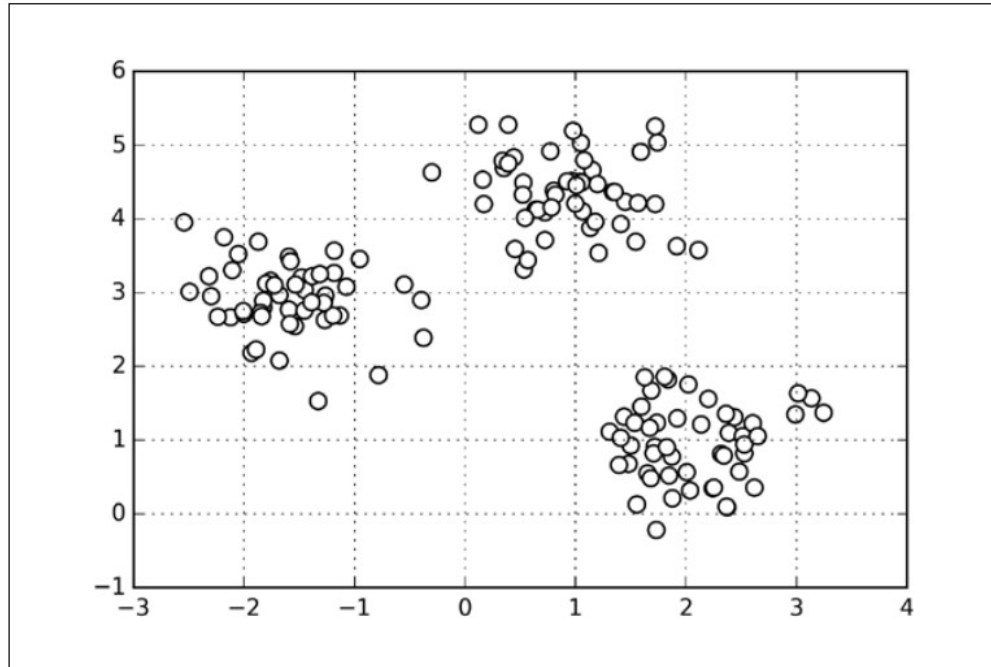
Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Learning goals and outline

Unsupervised classification

- 1) Understanding the k -means algorithm
- 2) Understanding the difference between unsupervised and supervised learning
- 3) Being able to build your own k -means algorithm

Unlabelled data



Optimisee

WITHIN-CLUSTER SUM OF SQUARED ERRORS

$$SSE = \sum_{i=1}^n \sum_{j=1}^k w^{(i, j)} \left\| \mathbf{x}^{(i)} - \boldsymbol{\mu}^{(j)} \right\|_2^2$$

i : sample index

j : cluster index

n : n samples

k : n clusters

$\boldsymbol{\mu}$: centroid for cluster j

$w^{(i, j)} = 1$, if $\mathbf{x}^{(i)}$ in cluster j

$w^{(i, j)} = 0$, if $\mathbf{x}^{(i)}$ *not* in cluster j

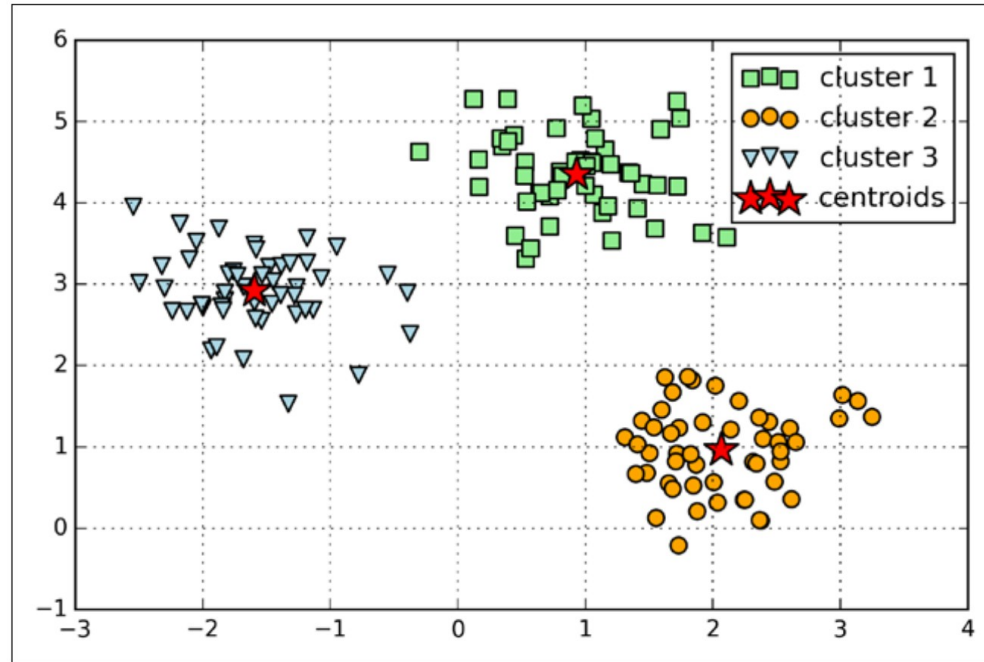
Running over each cluster

Running over each sample

Minimise distance from centroid

... but only if it is part of cluster

Labelled by k -means



Features and issues – k -means

- When converged, clusters may reveal patterns in data
- Some clusters may end up being empty
- k must be chosen *a priori* without any guiding information
 - elbow and silhouette plots may help
- May converge slowly or with vastly different results if *centroids* are chosen unluckily, e.g. too close to one another
 - k -means++ may help, i.e. it places initial centroids far from one another
- Clusters don't overlap
 - You can alter the “hardness” of the cluster

Summary

- You have learnt the basics of machine learning
- You have learnt to fit via gradient descent
- You have learnt how to optimise and do out-of-sample testing, thereby assessing the stability of models
 - e.g. by using **regularisation**, **feature selection (PCA)**, **standardisation**, choosing **hyperparameters** and using **cross-validation**
- You have learnt how to apply neural networks and unsupervised learning

Learning goals and outline

Wrap-up of course

1) Understanding how it all fits together

Did we learn what we should?

Academic regulations – objectives

Knowledge:

- demonstrate understanding of statistical techniques relying on the Generalised Linear Model
- demonstrate understanding of hierarchical modeling methods
- demonstrate understanding of basic machine learning concepts.

Academic regulations – objectives

Skills:

- build and evaluate models of hierarchically structured data
- integrate machine learning procedures in data analysis
- communicate analysis processes, results and interpretation.

Academic regulations – objectives

Competences:

- independently decide on data analysis methods, given a data set and a research question
- justify decision making when pre-processing messy data for data analysis.

Evaluation

https://go.blueja.io/RmuNdnx_rUeQH5n00xkj1g

**Also
available on
Brightspace
under
Assignments**



Thank you!

- for joining me on this ...
 - frustrating
 - challenging
 - time-demanding
- but hopefully also ...
 - rewarding
 - informative
 - challenging (!)

Journey towards
knowledge, skill and
competence

*Yes, I have very
few jokes in my
repertoire*



The only inspirational
picture you got in a
whole course:

<https://www.diygenius.com/self-education-will-make-you-a-fortune/>

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Next time

- Summarising machine learning
 - By Lydia

Reading questions

No more readings!